# Assignment 3: A Lagrangian Heuristic for UFL

1CM260 Large-scale Optimization for Transport & Mobility


## Yusuf Berkan Gokce

Student ID: 2372711


Quarter 1



Eindhoven, October 23, 2026

# 1 Introduction

The Uncapacitated Facility Location Problem (UFL) aims to determine which facilities to open and how to assign customers to them while minimizing total facility opening and transportation costs. The problem involves a trade-off between opening additional facilities and reducing customer assignment costs, making it a challenging combinatorial optimization problem.

This report implements a Lagrangian heuristic that relaxes the customer assignment constraints to obtain lower bounds, constructs feasible solutions to obtain upper bounds, and iteratively improves the multipliers using a Polyak-type subgradient method. The approach is evaluated on five benchmark instances (MO1–MO5), achieving relative optimality gaps of 2.26%–6.01%. The results are analyzed in terms of solution quality, convergence behavior, and opportunities for improvement.

# 2 Methodology

The Lagrangian heuristic relaxes the customer assignment constraints of the Uncapacitated Facility Location Problem (UFL). At each iteration, the relaxation provides a lower bound (LB), while a feasible solution constructed from the relaxed solution provides an upper bound (UB). The multipliers are iteratively updated to improve the lower bound.

## 2.1 Lagrangian Relaxation and Bound Construction

The assignment constraints $\sum_j x_{ij} = 1$ are relaxed using unrestricted multipliers $\lambda_i \in R$. The resulting Lagrangian dual function is

$$\theta(\boldsymbol{\lambda}) = \sum_i \lambda_i + \sum_j \min\left\{0,\ f_j + \sum_i \min(c_{ij} - \lambda_i, 0)\right\}, \tag{1}$$

where $f_j$ and $c_{ij}$ denote facility opening and transportation costs, respectively. The relaxed problem is separable by facility: facility $j$ is opened when its adjusted contribution is negative, and assignments are selected when their adjusted transportation costs are negative.

To restore feasibility, each customer is assigned to its cheapest open facility using the original transportation costs. If no facility is open, the facility minimizing $f_j + \sum_i c_{ij}$ is selected. The resulting feasible solution provides an upper bound.

## 2.2 Multiplier Update Rule and AI Assistance

ChatGPT (OpenAI) was consulted for the Lagrangian multiplier update formula. The complete prompt and response are provided in Appendix A.

The suggested approach was a Polyak-type subgradient ascent method. At iteration $k$, the subgradient and multiplier update are

$$g_i^{(k)} = 1 - \sum_j x_{ij}^{(k)}, \qquad \lambda_i^{(k+1)} = \lambda_i^{(k)} + \alpha_k g_i^{(k)}, \tag{2}$$

with step size

$$\alpha_k = \beta_k \frac{\max\{0, UB_{\text{best}} - \theta(\boldsymbol{\lambda}^{(k)})\}}{\sum_i (g_i^{(k)})^2}. \tag{3}$$

The update adjusts multipliers according to assignment violations. An unassigned customer has $g_i^{(k)} > 0$, increasing its multiplier and making assignment more attractive in the next relaxed problem. Conversely, a customer assigned multiple times has $g_i^{(k)} < 0$, decreasing its multiplier and discouraging excess assignments. The step size depends on the difference between the best feasible upper bound and the current Lagrangian lower bound, while normalization by the squared subgradient norm controls the update magnitude.

Since the Lagrangian dual function is concave but generally nonsmooth, subgradient updates do not guarantee improvement at every iteration. Consequently, the best lower bound is retained throughout the procedure, while step-size reduction helps limit oscillations around promising multiplier values.

The suggested formula was implemented independently. The multipliers are initialized as $\lambda_i^{(0)} = \min_j c_{ij}$, with $\beta_0 = 1.5$. To reduce oscillations, $\beta_k$ is halved after 20 consecutive iterations without improvement in the best lower bound. A zero subgradient is handled separately to avoid division by zero. No nonnegativity restriction is imposed on the multipliers because the relaxed constraints are equalities.

### 2.3 Termination Criteria

The heuristic maintains the best lower and upper bounds across iterations, ensuring that the recorded lower bound is nondecreasing and the upper bound is nonincreasing.

The algorithm terminates when the fractional optimality gap is at most $10^{-6}$ (equivalent to 0.0001%), the subgradient becomes zero, or the maximum of 5,000 iterations is reached. The relative optimality gap, expressed as a percentage, is calculated as

$$\text{Gap}(\%) = \frac{UB_{\text{best}} - LB_{\text{best}}}{\max(1, |UB_{\text{best}}|)} \times 100. \tag{4}$$

## 3 Results

The Lagrangian heuristic was evaluated on five benchmark instances (MO1–MO5), using a maximum of 5,000 iterations per instance. Table 1 reports the best Lagrangian lower bounds (LB), feasible upper bounds (UB), and relative optimality gaps.

**Table 1:** Computational results for the UFL benchmark instances.

| Instance | Best LB | Best UB | Gap (%) | Open Fac. | Time (s) |
|---|---|---|---|---|---|
| MO1 | 3230.55 | 3382.96 | 4.51 | 7 | 0.8841 |
| MO2 | 3779.39 | 3866.96 | 2.26 | 8 | 0.7955 |
| MO3 | 3978.02 | 4232.33 | 6.01 | 8 | 0.7979 |
| MO4 | 3932.94 | 4166.71 | 5.61 | 8 | 0.8316 |
| MO5 | 3590.19 | 3714.09 | 3.34 | 7 | 0.8112 |

All five instances produced feasible solutions, with relative optimality gaps ranging from 2.26% to 6.01%. MO2 achieved the smallest gap, while MO3 exhibited the largest. These gaps measure the separation between the best feasible solution and the Lagrangian lower bound, rather than the actual deviation from the unknown optimum.

The convergence plots in Appendix B show rapid initial improvements followed by prolonged stagnation. All instances reached the 5,000-iteration limit without satisfying the early stopping conditions.

The multiplier trajectories in Appendix C illustrate how the dual variables evolve, revealing pronounced initial oscillations followed by stabilization. The subgradient and step-size diagnostics in Appendix D provide further insight into this behavior by showing that assignment violations persist even as the Polyak step sizes approach zero. Together, these figures suggest that repeated step-size reductions suppress further multiplier adjustments rather than achieving complete satisfaction of the relaxed constraints. The appendices provide a more detailed examination of these convergence limitations.

Appendix E presents schematic network visualizations of the best feasible solutions, illustrating facility openings and customer assignments. These diagrams are not geographical maps, since actual facility and customer coordinates were not provided.

## 4 Conclusion

The Lagrangian heuristic produced feasible solutions for all five UFL instances, with optimality gaps between 2.26% and 6.01% and execution times below 0.9 seconds. While additional iterations improved some upper bounds, the lower bounds eventually stopped improving, suggesting limitations in the multiplier update procedure.

The multiplier and subgradient plots help explain this behavior. The multipliers change significantly during the first iterations but eventually become almost constant, even though some assignment constraints are violated in the relaxed problem. This happens because the step size is repeatedly reduced when the lower bound stops improving. As a result, the multiplier updates become too small to make further progress. A less aggressive step-size reduction strategy or alternative methods, such as bundle methods, could improve convergence.

The remaining optimality gaps depend on both the lower bounds and the quality of the feasible solutions. Therefore, improving feasible solutions through local search, such as opening, closing, or swapping facilities, could reduce the gaps. Comparing results with exact MILP solutions would show how close the heuristic solutions are to the optimum. Testing larger instances would help evaluate how well the algorithm performs on difficult problems.

Overall, Lagrangian relaxation is an efficient approach for solving UFL problems. However, improvements to both multiplier updates and feasible solution construction are needed to obtain tighter bounds and smaller optimality gaps.

# References

[1] Y. Beck, "Mip-based heuristics," Oct. 2026. Lecture notes, Large-Scale Optimization for Transport & Mobility, Eindhoven University of Technology, Department of Industrial Engineering and Innovation Sciences, October 1, 2026.

# A Generative AI Assistance: Multiplier Update Formula

ChatGPT (OpenAI) was consulted to obtain a mathematical update rule for the Lagrangian multipliers in the Uncapacitated Facility Location Problem. The following pages contain the complete conversation, including the prompt and the suggested formula.



**You said:**

I am working on a Master's assignment involving a **Lagrangian Heuristic for the Uncapacitated Facility Location Problem (UFL)**. We relax the customer assignment constraints using Lagrangian multipliers. The objective is to maximize the Lagrangian lower bound by iteratively updating these multipliers.
I am currently working on the multiplier update procedure and would like your help.
**My request:**
Please suggest a mathematically sound **update formula for the Lagrangian multipliers**, preferably using a standard subgradient optimization approach.
Explain:

1. The mathematical formula for updating each multiplier.
2. The intuition behind the update, particularly how it responds to customers being undersupplied or oversupplied.
3. How to select an appropriate step size for good convergence behavior.
4. Whether the multipliers have any restrictions, such as nonnegativity.
5. Important considerations regarding numerical stability, oscillations, and convergence.

**Important:** Provide mathematical formulas and explanations only, without any code or pseudocode. I will implement the suggested formula myself.
Prefer a practical, well-established approach suitable for an iterative Lagrangian heuristic.

**ChatGPT said:**

For the Uncapacitated Facility Location Problem (UFL), I recommend using a **Polyak-type subgradient method** with an adaptive step size. This is a well-established approach for maximizing Lagrangian lower bounds and is particularly suitable for iterative Lagrangian heuristics.

### 1. Mathematical formulation

Consider the standard UFL formulation:

$$\min \sum_{i \in I} f_i y_i + \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij}$$

subject to the customer assignment constraints:

$$\sum_{i \in I} x_{ij} = 1, \qquad \forall j \in J$$

where $x_{ij}$ indicates whether customer $j$ is assigned to facility $i$, and $y_i$ indicates whether facility $i$ is open.

Relaxing the assignment constraints with multipliers $\lambda_j$, using the convention $1 - \sum_i x_{ij}$, gives:

$$L(x, y, \lambda) = \sum_i f_i y_i + \sum_i \sum_j c_{ij} x_{ij} + \sum_j \lambda_j \left(1 - \sum_i x_{ij}\right)$$

Rearranging:

$$L(x, y, \lambda) = \sum_i f_i y_i + \sum_i \sum_j (c_{ij} - \lambda_j) x_{ij} + \sum_j \lambda_j$$

The Lagrangian dual function is:

$$q(\lambda) = \min_{(x,y) \in X} L(x, y, \lambda)$$

where $X$ contains the constraints that have not been relaxed.

The objective is:

$$\max_{\lambda \in \mathbb{R}^{|J|}} q(\lambda)$$

## 2. Subgradient update formula

At iteration $k$, solve the Lagrangian subproblem to obtain $x_{ij}^{(k)}$.

The subgradient component for customer $j$ is:

$$g_j^{(k)} = 1 - \sum_{i \in I} x_{ij}^{(k)}$$

Then update the multiplier using:

$$\lambda_j^{(k+1)} = \lambda_j^{(k)} + \alpha_k g_j^{(k)}$$

Equivalently:

$$\lambda_j^{(k+1)} = \lambda_j^{(k)} + \alpha_k \left(1 - \sum_i x_{ij}^{(k)}\right)$$

Here, $\alpha_k > 0$ is the step size.

Strictly speaking, because $q(\lambda)$ is concave, $g^{(k)}$ is a *supergradient* of the dual function. The update is commonly called subgradient ascent in Lagrangian optimization.

### 3. Intuition: undersupply and oversupply

The update automatically adjusts the multipliers based on the assignment constraint violations.

| Customer assignment | Subgradient | Multiplier update |
|---|---|---|
| Undersupplied: $\sum_i x_{ij} = 0$ | $g_j = 1$ | Increase $\lambda_j$ |
| Correctly supplied: $\sum_i x_{ij} = 1$ | $g_j = 0$ | No change |
| Oversupplied: $\sum_i x_{ij} > 1$ | $g_j < 0$ | Decrease $\lambda_j$ |

Recall that the modified assignment cost is:

$$\tilde{c}_{ij} = c_{ij} - \lambda_j$$

Therefore:

- **Undersupplied customer:** Increasing $\lambda_j$ reduces the modified assignment cost, encouraging facilities to serve this customer.
- **Oversupplied customer:** Decreasing $\lambda_j$ increases the modified assignment cost, discouraging excessive assignments.
- **Correctly supplied customer:** No adjustment is necessary for that customer at the current iteration.

This feedback mechanism pushes the relaxed solution toward satisfying the original assignment constraints.

### 4. Step-size selection: Polyak's method

Rather than using a constant step size, I recommend:

$$\alpha_k = \theta_k \frac{UB - q(\lambda^{(k)})}{\sum_{j \in J} (g_j^{(k)})^2}$$

where:

- $UB$ is the objective value of a feasible UFL solution (an upper bound on the optimal minimization cost).
- $q(\lambda^{(k)})$ is the current Lagrangian lower bound.
- $\sum_j (g_j^{(k)})^2$ measures the total squared assignment violation.
- $\theta_k$ is a step-size scaling parameter.

Combining this with the multiplier update gives:

$$\lambda_j^{(k+1)} = \lambda_j^{(k)} + \theta_k \frac{UB - q(\lambda^{(k)})}{\sum_{\ell \in J} (g_\ell^{(k)})^2} \left(1 - \sum_i x_{ij}^{(k)}\right)$$

This is the main formula I recommend implementing.

#### Choosing $\theta_k$

A practical starting point is:

$$\theta_0 = 1.5$$

with:

$$0 < \theta_k < 2$$

A commonly used heuristic is to reduce $\theta_k$ when the best lower bound has not improved for several iterations.

For example:

$$\theta_{k+1} = \begin{cases} \theta_k/2 & \text{after } N \text{ consecutive iterations without improvement} \\ \theta_k & \text{otherwise} \end{cases}$$

You could start with $N = 20$, although the appropriate value depends on the problem size.

The Polyak step uses the current duality gap to control update magnitude. A larger gap permits larger steps, while a smaller gap generally produces more conservative updates.

**Important theoretical distinction:** The classical Polyak convergence result uses the optimal dual value $q^*$, rather than an arbitrary primal upper bound $UB$. Replacing $q^*$ with $UB$ is a standard practical heuristic, but it does not automatically provide the same convergence guarantees, especially when the Lagrangian duality gap is positive.

### 5. Restrictions on multipliers

Because the relaxed assignment constraints are equalities,

$$\sum_i x_{ij} = 1,$$

the associated multipliers are **unrestricted in sign**:

$$\lambda_j \in \mathbb{R}$$

Therefore, you should not impose nonnegativity on the multipliers.

In particular, avoid applying:

$$\lambda_j^{(k+1)} = \max(0, \lambda_j^{(k)} + \alpha_k g_j^{(k)})$$

unless you have explicitly reformulated the relaxation using inequality constraints with the corresponding sign convention.

For your equality relaxation, no projection is necessary.

### 6. Numerical stability and convergence

There are several important implementation considerations.

**Oscillations.** Subgradient methods often oscillate near the dual optimum because the Lagrangian dual function is piecewise linear and nonsmooth. Reducing $\theta_k$ after stagnation can help stabilize the iterations.

**Zero subgradient.** If:

$$\sum_j (g_j^{(k)})^2 = 0,$$

the Polyak formula is undefined. Do not divide by zero. Instead, stop the multiplier update. If the Lagrangian subproblem was solved exactly, a zero subgradient indicates that the current multipliers maximize the dual function.

**Best lower bound.** The current lower bound need not improve monotonically. Maintain:

$$LB_{\text{best}}^{(k)} = \max_{t \le k} q(\lambda^{(t)})$$

Use this best lower bound to evaluate progress, rather than assuming every iteration improves the objective.

**Feasible upper bound.** Update $UB$ whenever your primal heuristic discovers a better feasible solution. A tighter upper bound usually improves the practical step-size estimate.

**Stopping criteria.** Possible criteria include a maximum iteration count, a sufficiently small primal-dual gap,

$$\frac{UB - LB_{\text{best}}}{\max(1, |UB|)} \le \varepsilon,$$

or prolonged stagnation after several step-size reductions.

If a guaranteed asymptotic convergence approach is more important than practical speed, a diminishing step-size sequence can be used instead:

$$\alpha_k = \frac{a}{(k+1)^p}, \qquad \frac{1}{2} < p \leq 1, \quad a > 0.$$

This satisfies the standard conditions:

$$\sum_{k=0}^{\infty} \alpha_k = \infty, \qquad \sum_{k=0}^{\infty} \alpha_k^2 < \infty.$$

Under the usual bounded-subgradient and finite-optimum assumptions, these conditions support convergence of the best dual objective value.

## Final recommendation

For your Master's assignment, I would use the following configuration:

| Component | Recommendation |
|---|---|
| Update method | Polyak-type subgradient ascent |
| Initial scaling parameter | $\theta_0 = 1.5$ |
| Step-size reduction | Halve after 20 iterations without best-bound improvement |
| Multiplier restrictions | Unrestricted, $\lambda_j \in \mathbb{R}$ |
| Lower-bound tracking | Best value across all iterations |
| Stopping | Iteration limit, small primal-dual gap, or prolonged stagnation |

The complete update is:

$$\lambda_j^{(k+1)} = \lambda_j^{(k)} + \theta_k \frac{UB - LB_k}{\sum_{\ell \in J} \left(1 - \sum_i x_{i\ell}^{(k)}\right)^2} \left(1 - \sum_i x_{ij}^{(k)}\right)$$

where $LB_k = q(\lambda^{(k)})$ is the **current** Lagrangian lower bound, not the historical best bound.

This is a practical and well-established multiplier update for a UFL Lagrangian heuristic.

# B Convergence of Lower and Upper Bounds

Figures 1–5 show the progression of the best Lagrangian lower bound and best feasible upper bound over 5,000 iterations for each benchmark instance. The plots illustrate the convergence behavior and the remaining optimality gaps.

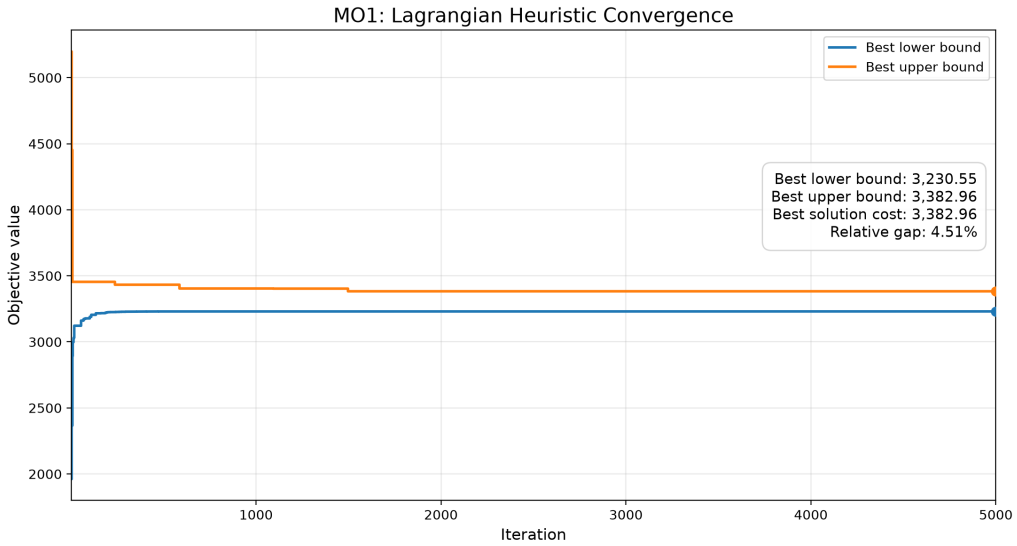



**Figure 1:** Convergence of lower and upper bounds for MO1.

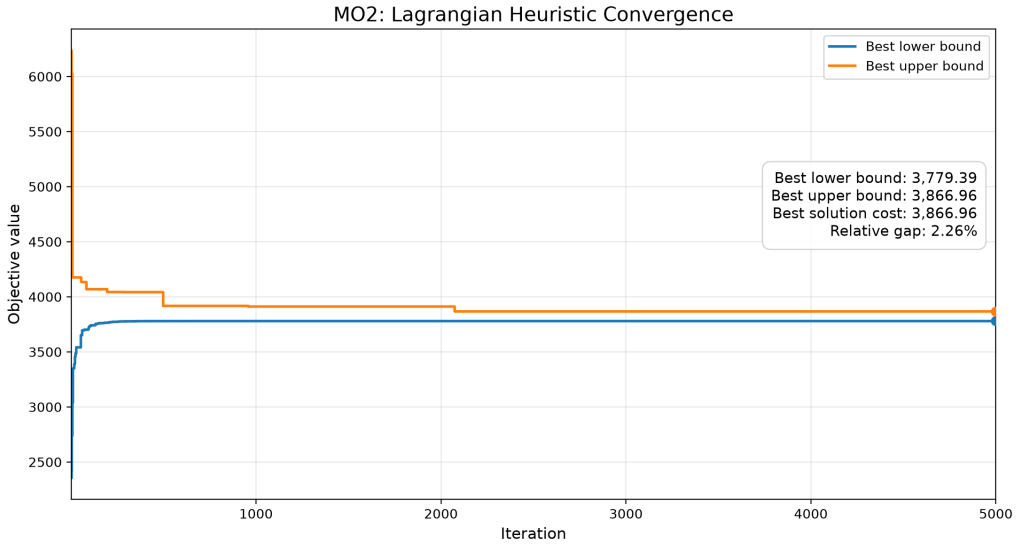



**Figure 2:** Convergence of lower and upper bounds for MO2.

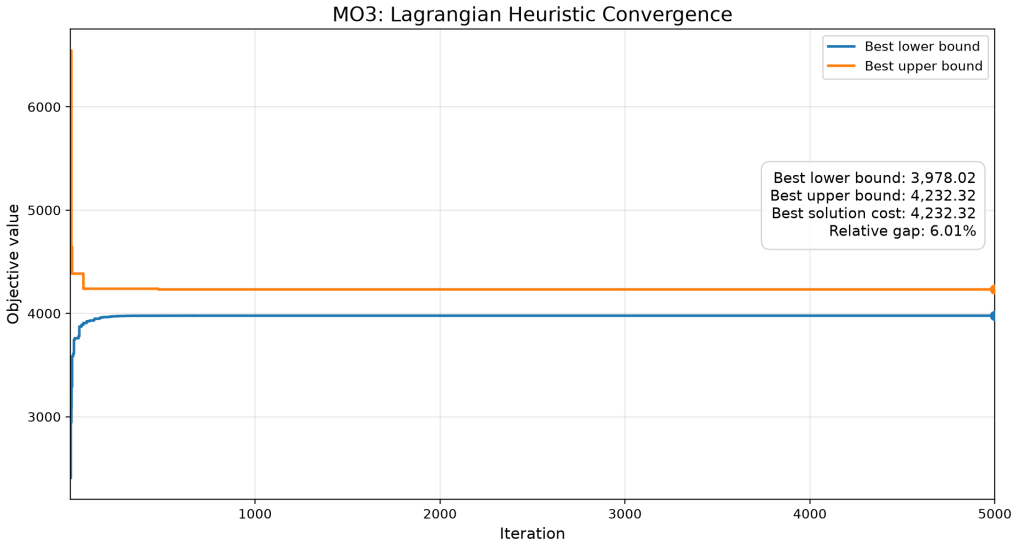



**Figure 3:** Convergence of lower and upper bounds for MO3.

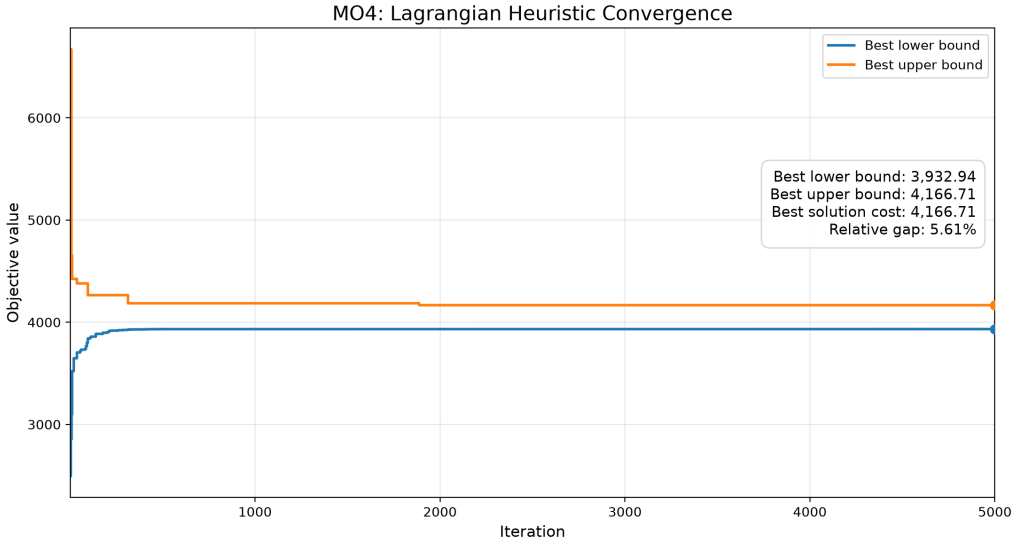



**Figure 4:** Convergence of lower and upper bounds for MO4.

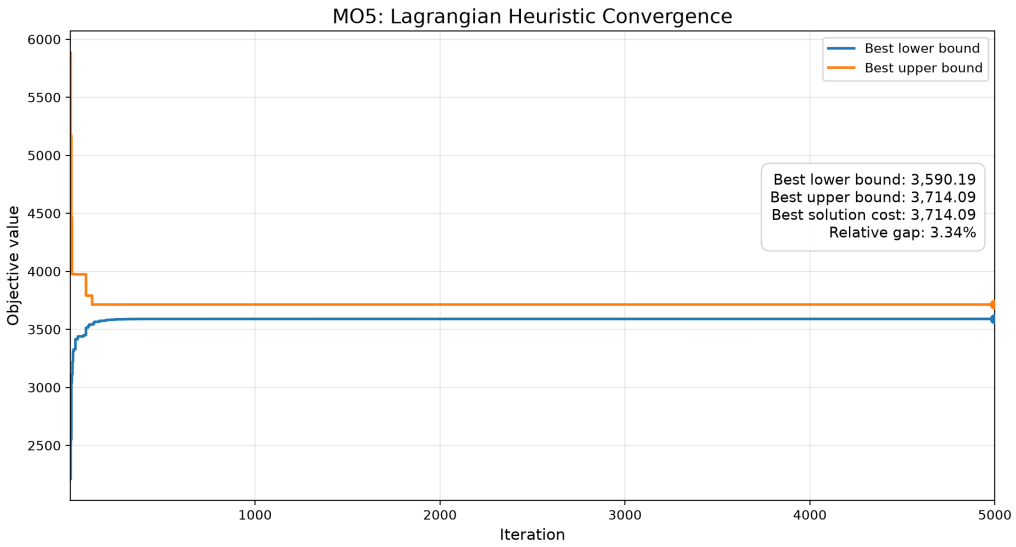



**Figure 5:** Convergence of lower and upper bounds for MO5.

# C Lagrangian Multiplier Evolution

The following figures illustrate the evolution of eight Lagrangian multipliers for each benchmark instance over 5,000 iterations. The customers were selected based on the highest standard deviation of their multiplier values across iterations, highlighting those experiencing the largest fluctuations. Each multiplier $\lambda_i$ corresponds to a customer's relaxed assignment constraint and influences the attractiveness of assigning that customer to a facility.

Across all five instances, the multiplier trajectories exhibit pronounced oscillations during the initial iterations. These fluctuations arise from the subgradient update rule, where multipliers increase when customers are unassigned and decrease when customers are assigned to multiple facilities in the relaxed solution. Large initial step sizes can cause the algorithm to overcorrect assignment violations, repeatedly switching between undersupply and oversupply. For example, in MO1, the multiplier associated with customer 94 initially fluctuates substantially before approaching a value of approximately 80. Similar behavior is observed across the other instances, although the magnitude and direction of the fluctuations differ between customers.

Following the initial oscillations, the multiplier trajectories gradually stabilize, generally within the first few hundred iterations. This behavior is consistent with the adaptive step-size strategy, which halves the parameter $\beta_k$ after 20 consecutive iterations without improvement in the best lower bound. As $\beta_k$ decreases, the Polyak step size becomes smaller, reducing the magnitude of subsequent multiplier adjustments. Consequently, the trajectories become nearly horizontal for much of the remaining 5,000 iterations, indicating that the multipliers undergo increasingly small changes rather than continuing to explore substantially different regions of the dual solution space.

An important observation is that multiplier stabilization does not necessarily imply convergence to the optimal Lagrangian dual solution. Although the trajectories become nearly constant, the subgradient diagnostics in Appendix D show that the subgradient norms remain nonzero throughout the final iterations. This indicates that the relaxed customer assignment constraints are not all satisfied by the selected Lagrangian solutions. Nevertheless, the multiplier updates become negligible because repeated reductions of $\beta_k$ drive the step sizes towards zero. Thus, the apparent convergence of the multiplier trajectories is more consistent with numerical stagnation than with a verified dual optimum.

These findings suggest that the current step-size reduction strategy successfully suppresses early oscillations but may become overly conservative. Alternative strategies, such as less aggressive step-size reductions or more stable dual optimization methods, including bundle methods, could potentially maintain progress for longer. However, additional experiments would be necessary to determine whether these modifications improve the resulting lower bounds.

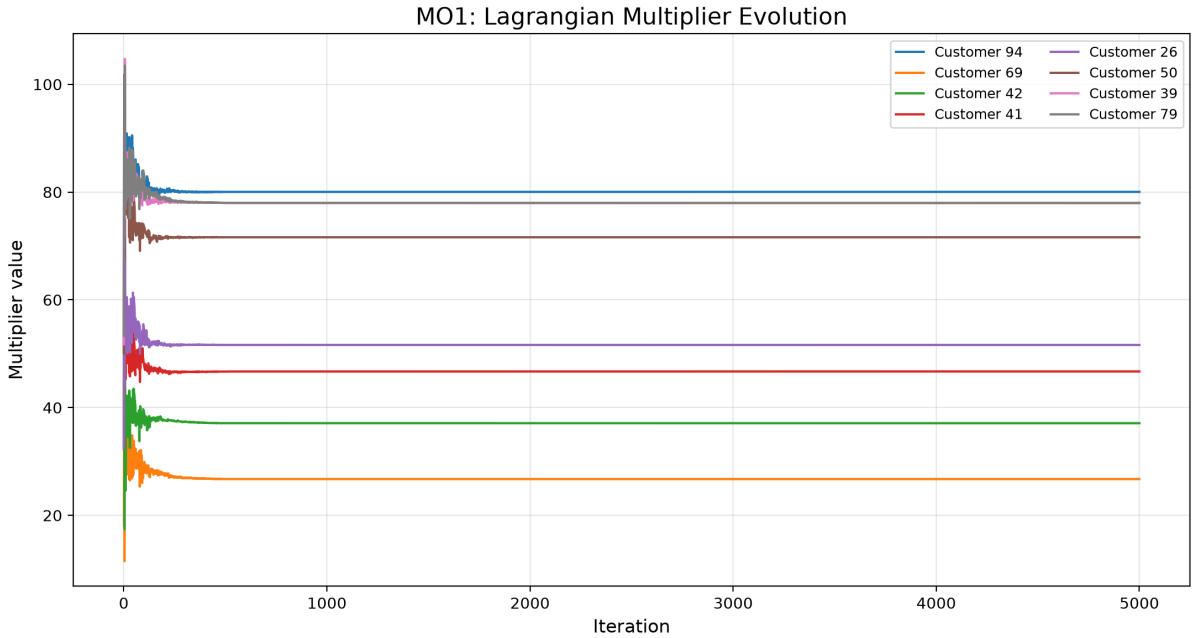



**Figure 6:** Lagrangian multiplier evolution for MO1.

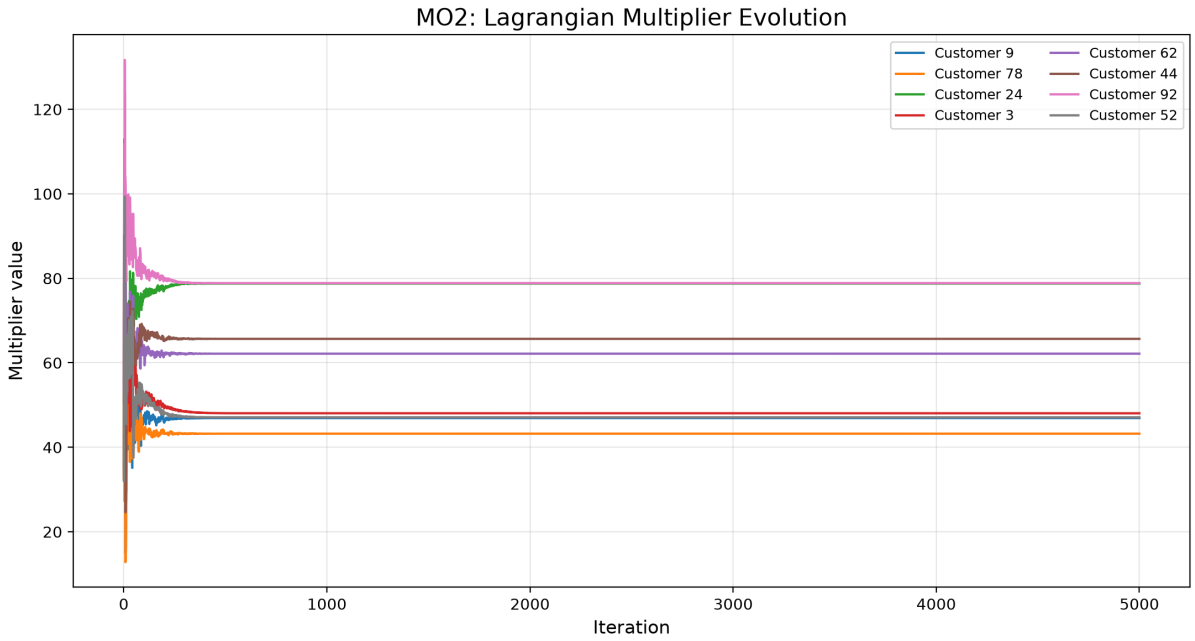



**Figure 7:** Lagrangian multiplier evolution for MO2.

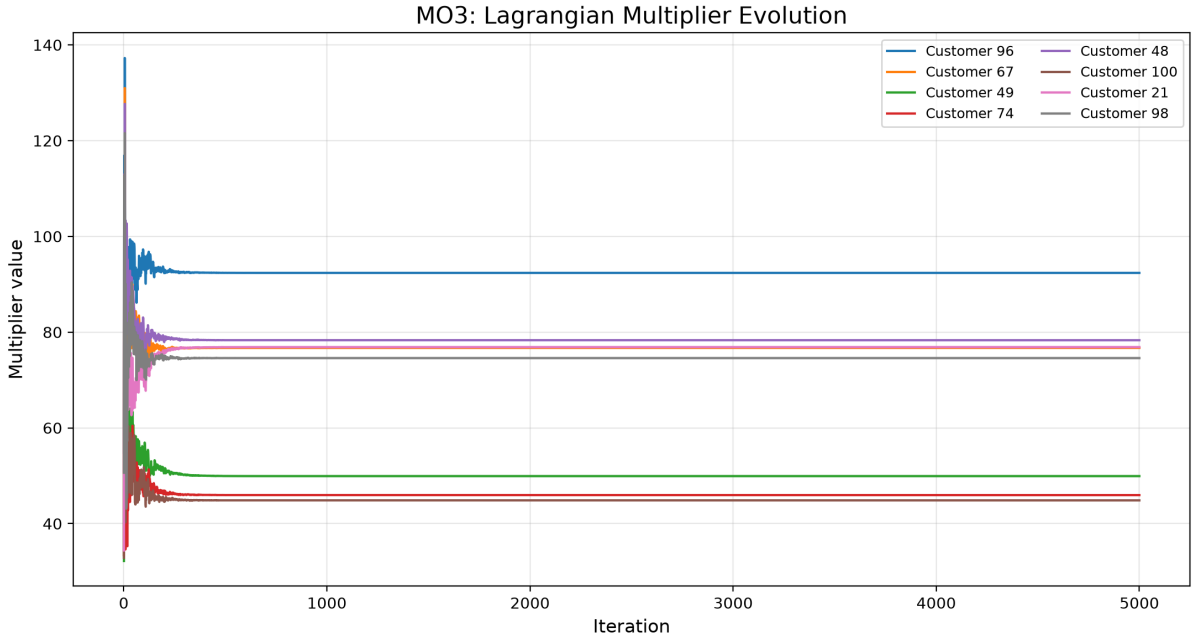



**Figure 8:** Lagrangian multiplier evolution for MO3.

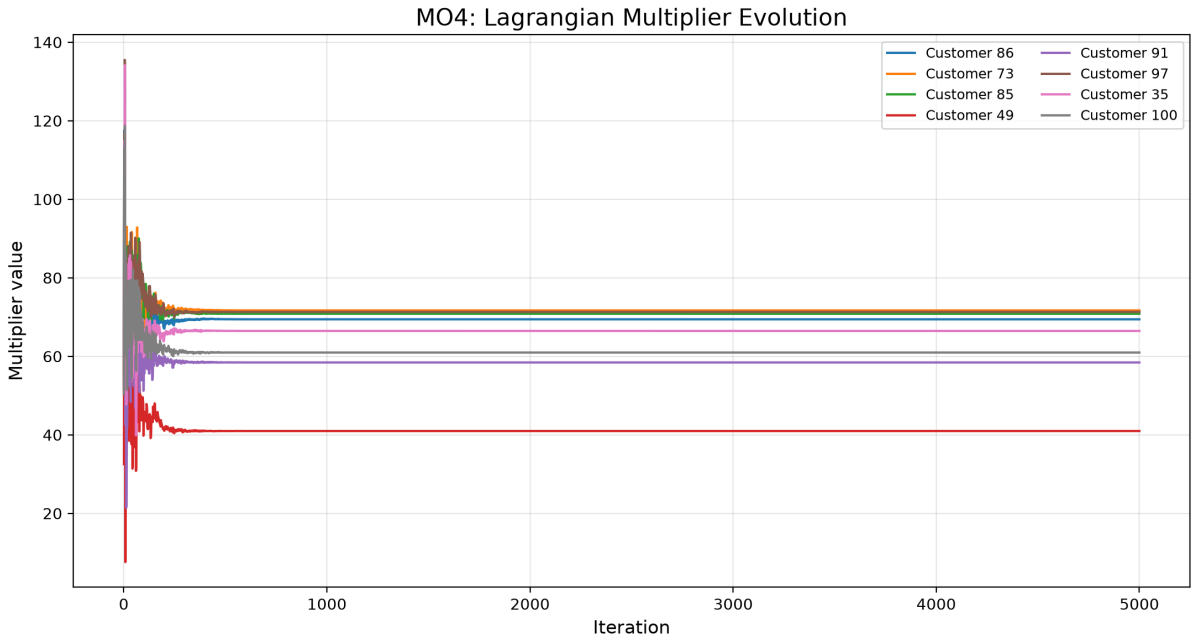



**Figure 9:** Lagrangian multiplier evolution for MO4.

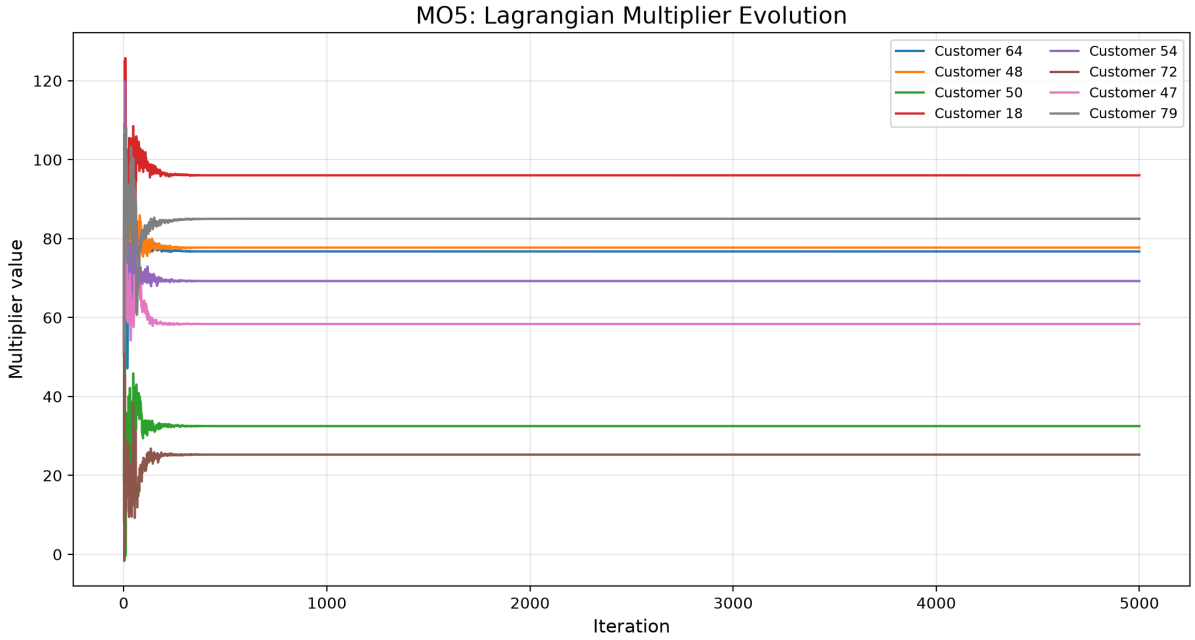



**Figure 10:** Lagrangian multiplier evolution for MO5.

# D Subgradient and Step-Size Diagnostics

The following figures illustrate the evolution of the subgradient norm and Polyak step size over 5,000 iterations for each benchmark instance. The subgradient norm, $\|g^{(k)}\|_2$, measures the aggregate violation of the relaxed customer assignment constraints, while the step size, $\alpha_k$, controls how strongly the Lagrangian multipliers respond to these violations. Together, these quantities provide insight into the behavior of the multiplier update procedure and the causes of convergence stagnation.

Across all five instances, the subgradient norms exhibit large initial fluctuations, reflecting substantial changes in the relaxed customer assignments. During these early iterations, the relatively large Polyak step sizes produce aggressive multiplier adjustments, which can cause customers to alternate between being unassigned and assigned to multiple facilities. As the algorithm progresses, both the subgradient fluctuations and step sizes decrease considerably, indicating that the multiplier updates become less aggressive.

An important observation is that the subgradient norms do not converge to zero. For example, in MO1, the norm initially exceeds 700 before decreasing substantially and eventually stabilizing at a positive value. This indicates that some customer assignment constraints remain violated in the relaxed solution. Although the magnitude of these violations becomes much smaller, the algorithm does not eliminate them entirely.

The Polyak step sizes decrease rapidly during the initial iterations and eventually become extremely small. This behavior is reinforced by the adaptive reduction of $\beta_k$, which is halved after 20 consecutive iterations without improvement in the best lower bound. Consequently, the multiplier adjustments become negligible even when the subgradient remains nonzero. The nearly constant subgradient norms during the later iterations suggest that the relaxed assignment decisions are no longer changing meaningfully.

These observations help explain the prolonged stagnation visible in the convergence plots. While the step-size reduction strategy effectively limits the large oscillations observed initially, repeated reductions eventually restrict the algorithm's ability to explore alternative multiplier values. The resulting behavior is consistent with numerical stagnation rather than verified convergence to the optimal Lagrangian dual solution. A nonzero subgradient alone does not rule out dual optimality for a nonsmooth function, but the remaining lower–upper bound gaps show that optimality has not been certified.

Overall, the diagnostics suggest that the effectiveness of the heuristic is limited partly by its step-size management. Less aggressive reductions, alternative step-size schedules, or more stable dual optimization methods, such as bundle methods, could potentially improve the lower bounds. Further computational experiments would be required to evaluate these alternatives.

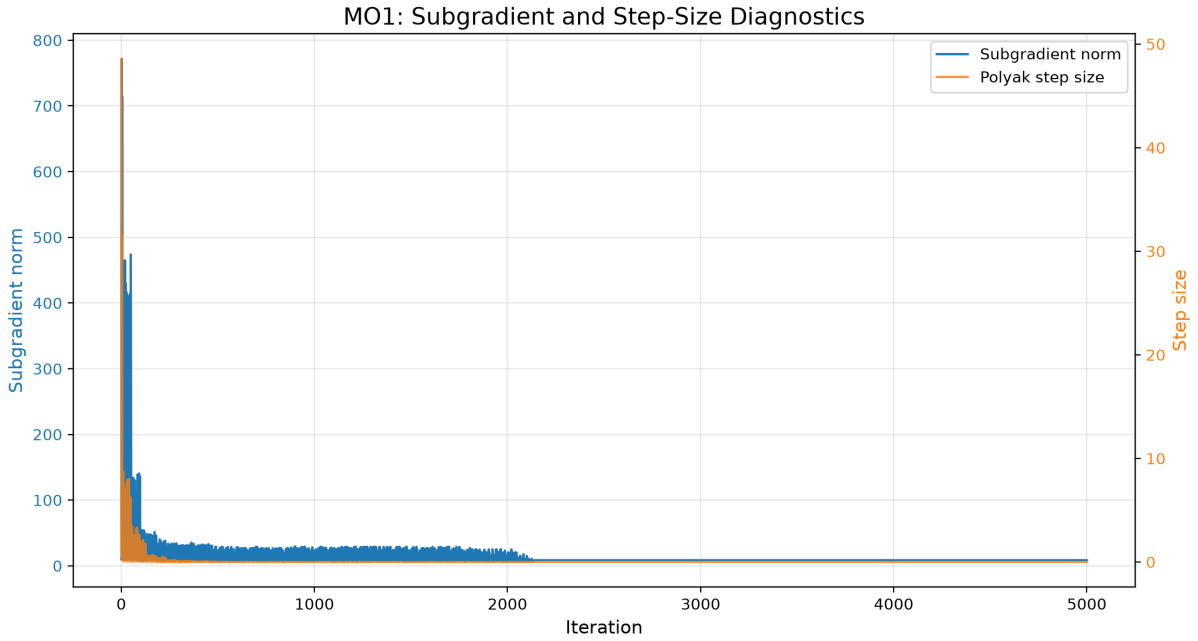



**Figure 11:** Subgradient norm and Polyak step size for MO1.

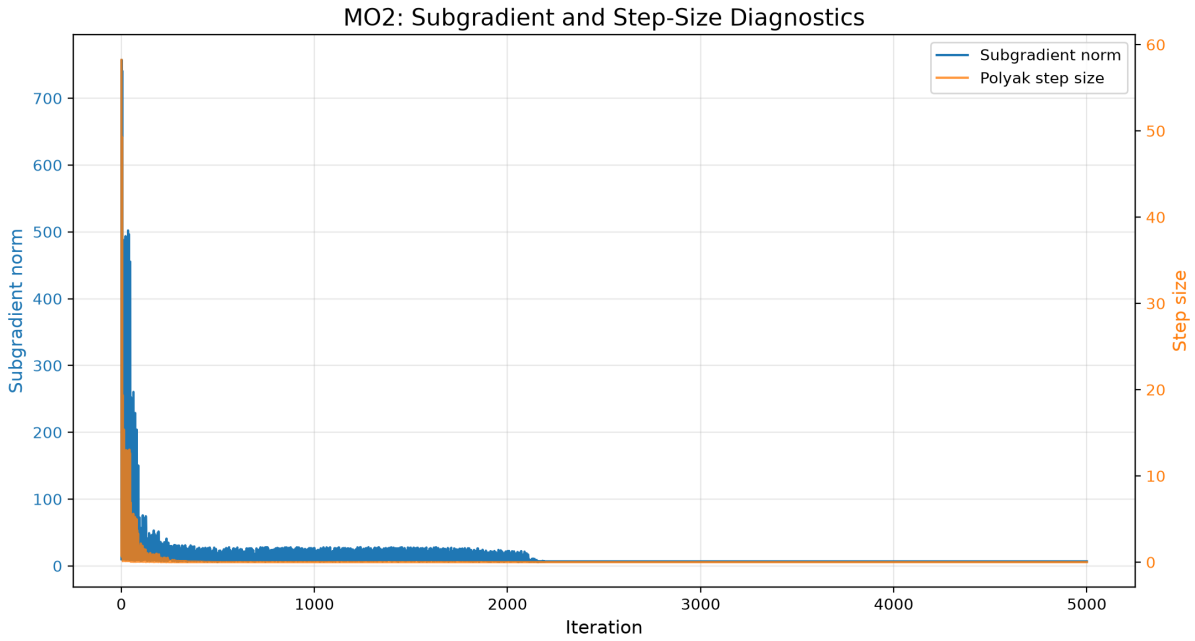



**Figure 12:** Subgradient norm and Polyak step size for MO2.

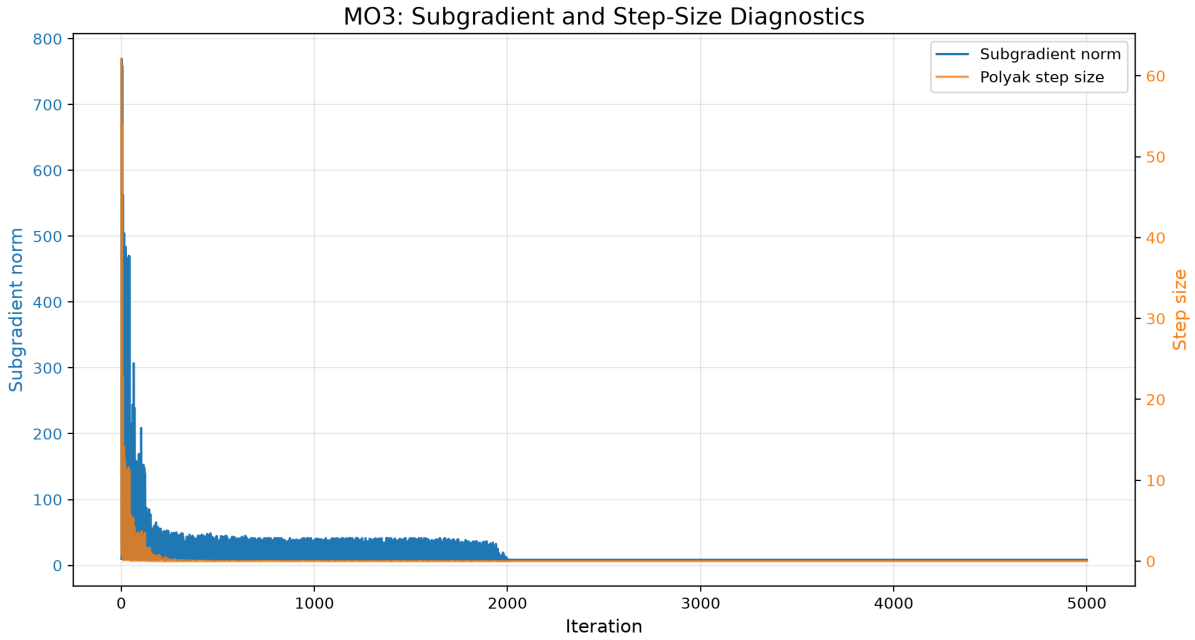



**Figure 13:** Subgradient norm and Polyak step size for MO3.

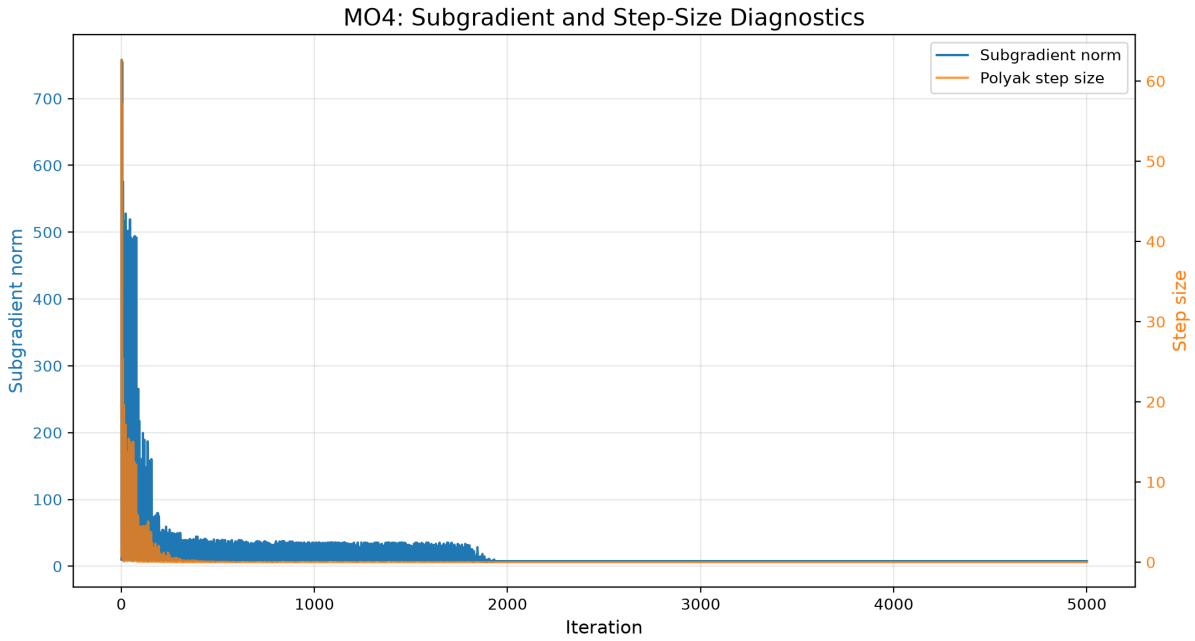



**Figure 14:** Subgradient norm and Polyak step size for MO4.

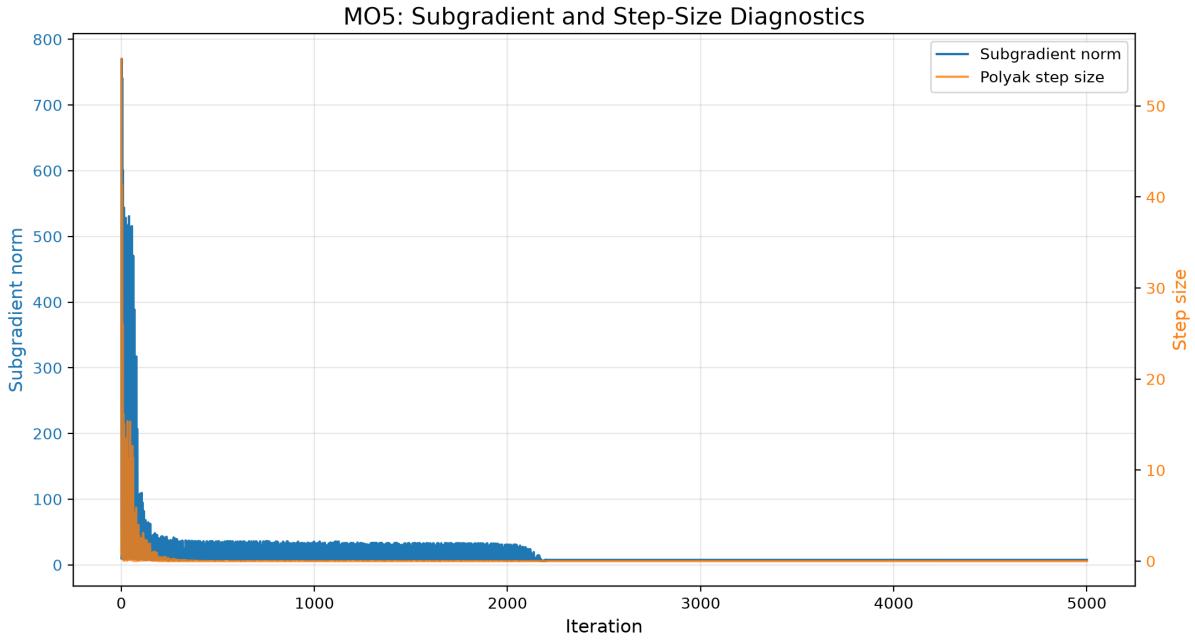



**Figure 15:** Subgradient norm and Polyak step size for MO5.

# E Facility–Customer Network Visualizations

Figures 16–20 illustrate the best feasible solutions obtained for the five benchmark instances. Open facilities and their assigned customers are shown together with the corresponding assignment connections.

These visualizations are schematic rather than geographical. Since the benchmark instances do not provide actual coordinates for facilities or customers, their positions were generated solely for visualization purposes. The diagrams therefore represent the assignment structure of the computed solutions, not real distances or geographical locations.

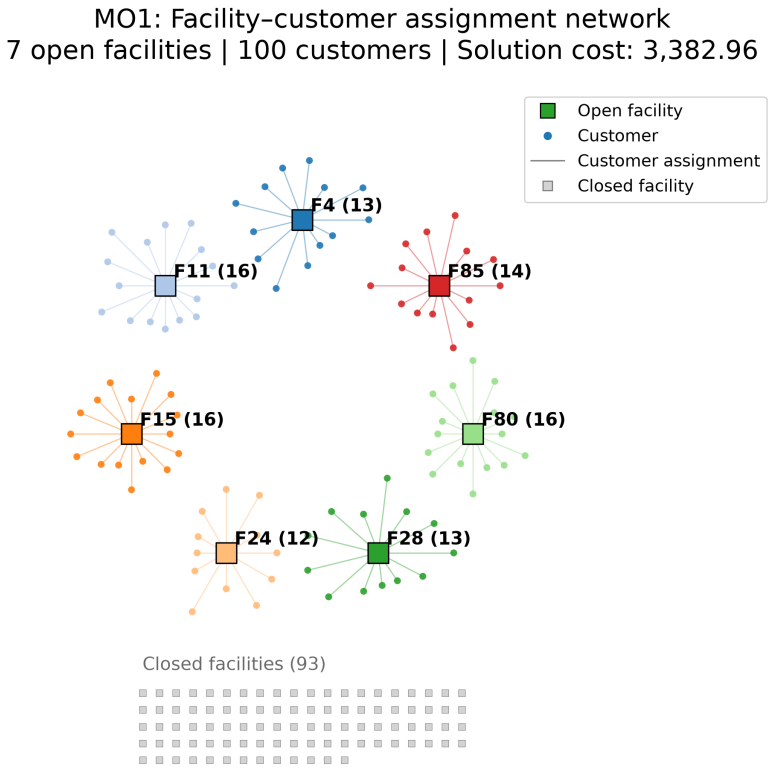



**Figure 16:** Schematic facility–customer assignment network for MO1.

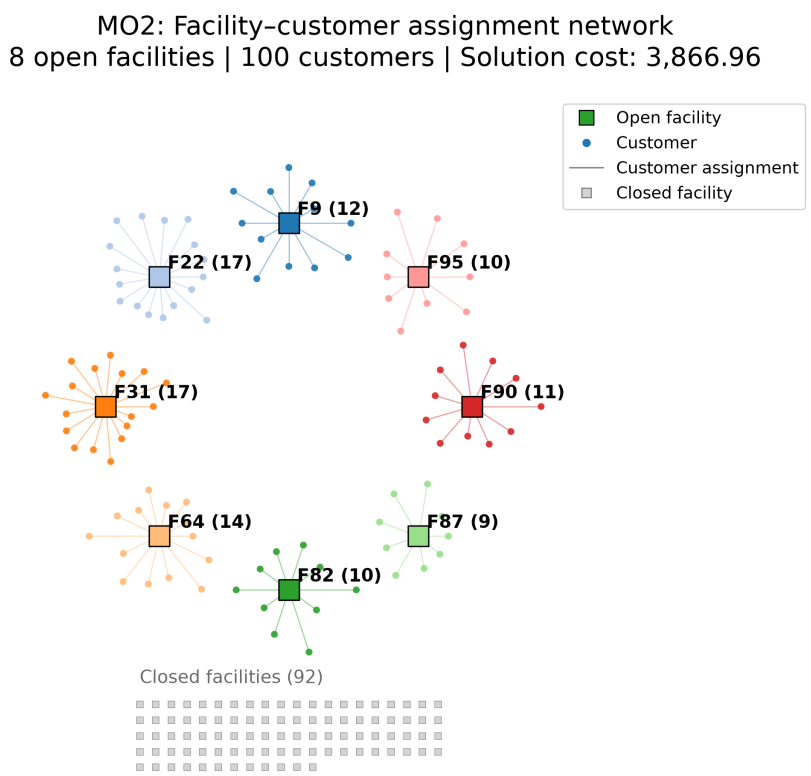



**Figure 17:** Schematic facility–customer assignment network for MO2.

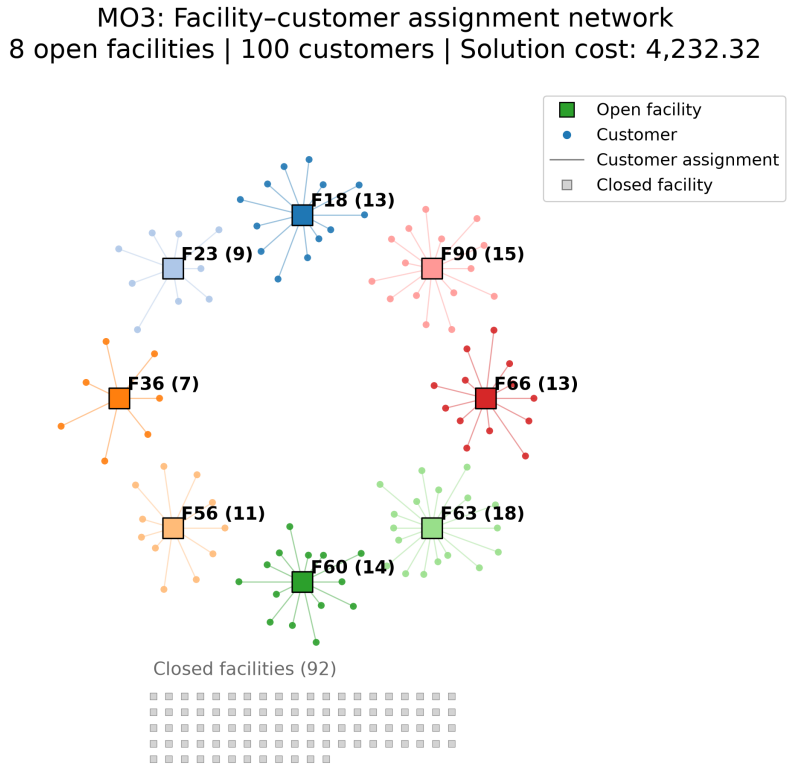



**Figure 18:** Schematic facility–customer assignment network for MO3.

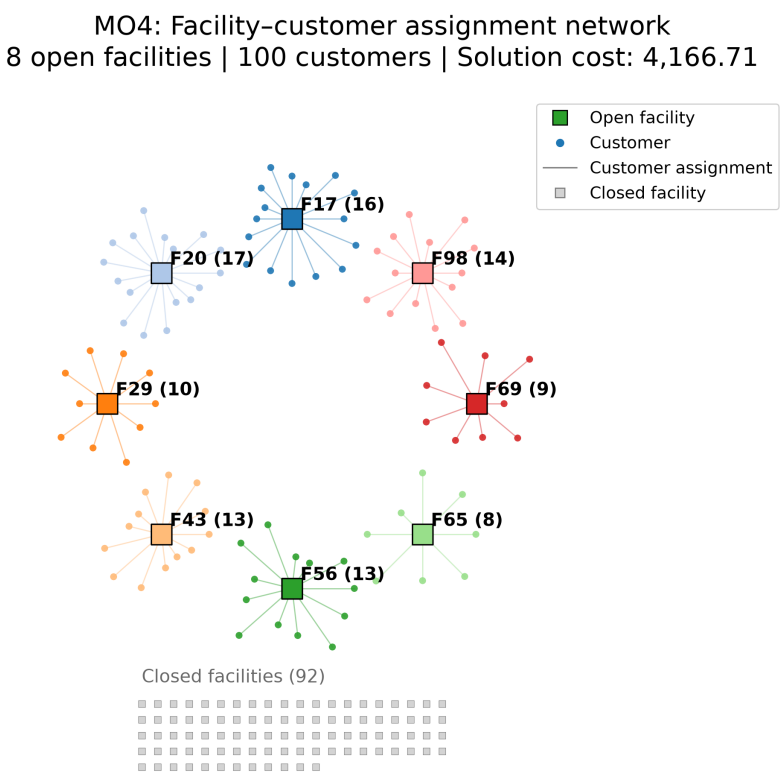



**Figure 19:** Schematic facility–customer assignment network for MO4.

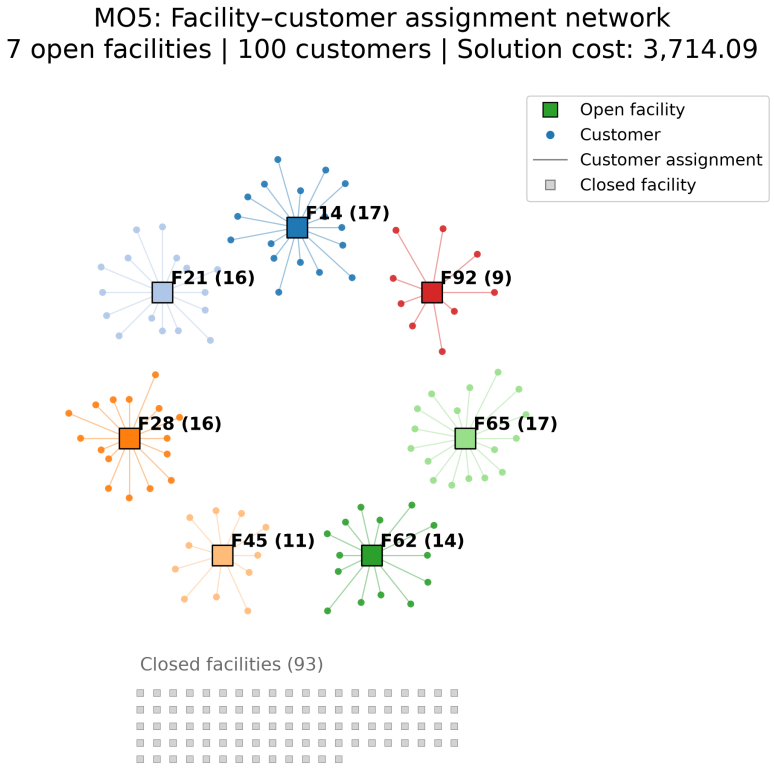



**Figure 20:** Schematic facility–customer assignment network for MO5.

# F Python Source Code

This appendix contains the complete Python implementation of the Lagrangian heuristic for the Uncapacitated Facility Location Problem (UFL), together with the script used to execute the computational experiments.

## F.1 `UFL.py` – Lagrangian Heuristic Implementation

The file `UFL.py` contains the problem and solution classes, the Lagrangian relaxation, the feasible solution construction, the subgradient multiplier update, and the iterative heuristic.

**Listing 1:** Complete implementation of the UFL Lagrangian heuristic

```python
# -*- coding: utf-8 -*-
"""
@author: Original template by Rolf van Lieshout and Krissada Tundulyasaree
"""
import numpy as np


class UFL_Problem:
    """
    Class that represent a problem instance of the Uncapcitated Facility Location Problem

    Attributes
    ----------
    f : numpy array
        the yearly fixed operational costs of all facilities
    c : numpy 2-D array (matrix)
        the yearly transportation cost of delivering all demand from markets to facilities
    n_markets : int
        number of all markets.
    n_facilities : int
        number of all available locations.
    """

    def __init__(self, f, c, n_markets, n_facilities):

        self.f = f
        self.c = c
        self.n_markets = n_markets
        self.n_facilities = n_facilities

    def __str__(self):
        return f" Uncapacitated Facility Location Problem: {self.n_markets} markets, {self.n_facilities} facilities"

    def readInstance(fileName):
        """
        Read the instance fileName

        Parameters
        ----------
        fileName : str
            instance name in the folder Instance.

        Returns
        -------
        UFL Object

        """
        # Read filename
        f = open(f"Instances/{fileName}")
        n_line = 0
        n_markets = 0
        n_facilities = 0
        n_row = 0
        for line in f.readlines():
            asList = line.replace(" ", "_").split("_")
            if line:
                if n_line == 0:
                    n_markets = int(asList[0])
                    n_facilities = int(asList[1])
                    f_j = np.empty(n_facilities)
                    c_ij = np.empty((n_markets, n_facilities))
                elif n_line <= n_facilities:  # For facilities
                    index = n_line - 1
                    f_j[index] = asList[1]
                else:
                    if len(asList) == 1:
                        n_row += 1
                        demand_i = float(asList[0])
                        n_column = 0
                    else:
                        for i in range(len(asList) - 1):
                            c_ij[n_row - 1, n_column] = (
```

```
                                demand_i * float(asList[i])
                            )
                            n_column += 1
                n_line += 1
        f.close()
        return UFL_Problem(f_j, c_ij, n_markets, n_facilities)


class UFL_Solution:
    """
    Class that represent a solution to the Uncapcitated Facility Location Problem

    Attributes
    ----------
    y : numpy array
        binary array indicating whether facilities are open
    x : numpy 2-D array (matrix)
        fraction of demand from markets sourced from facilities
    instance: UFL_Problem
        the problem instance
    """

    def __init__(self, y, x, instance):
        self.y = y
        self.x = x
        self.instance = instance

    def isFeasible(self):
        """
        Method that checks whether the solution is feasible

        Returns true if feasible, false otherwise
        """
        # Check dimensions
        if self.y.shape != (self.instance.n_facilities,):
            return False

        if self.x.shape != (self.instance.n_markets, self.instance.n_facilities):
            return False

        # Check binary facility decisions
        if not np.all((self.y == 0) | (self.y == 1)):
            return False

        # Check assignment bounds: 0 <= x_ij <= y_j
        if np.any(self.x < -1e-9) or np.any(self.x > self.y + 1e-9):
            return False

        # Check that each customer is fully supplied
        if not np.allclose(np.sum(self.x, axis=1), 1):
            return False

        return True


    def getCosts(self):
        """
        Method that computes and returns the costs of the solution
        """
        transportation_cost = np.sum(self.instance.c * self.x)
        opening_cost = np.sum(self.instance.f * self.y)

        return float(transportation_cost + opening_cost)

class LagrangianHeuristic:
    """
    Class used for the Lagrangian Heuristic

    Attributes
    ----------
    instance : UFL_Problem
        the problem instance
    """

    def __init__(self,instance):
        self.instance = instance

        # Parameters for the Polyak subgradient update
        self.beta = 1.5
        self.bestUB = float("inf")

    def computeTheta(self,labda):
        """
        Method that, given an array of Lagrangian multipliers computes and returns
        the optimal value of the Lagrangian problem
        """

```

```
        # Adjust transportation costs using customer multipliers
        adjusted_costs = self.instance.c - labda[:, np.newaxis]

        # Only negative adjusted costs improve the relaxed objective
        negative_costs = np.minimum(adjusted_costs, 0)

        # Cost contribution if each facility is opened
        opening_contributions = self.instance.f + np.sum(
            negative_costs, axis=0
        )

        # Compare opening each facility against keeping it closed
        facility_contributions = np.minimum(opening_contributions, 0)

        # Lagrangian lower bound
        theta = np.sum(labda) + np.sum(facility_contributions)

        return float(theta)


    def computeLagrangianSolution(self,labda):
        """
        Method that, given an array of Lagrangian multipliers computes and returns
        the Lagrangian solution (as a UFL_Solution)
        """

        # Adjust transportation costs using customer multipliers
        adjusted_costs = self.instance.c - labda[:, np.newaxis]

        # Calculate the cost contribution if each facility opens
        opening_contributions = self.instance.f + np.sum(
            np.minimum(adjusted_costs, 0), axis=0
        )

        # Open facilities only when their contribution is negative
        y = (opening_contributions < 0).astype(int)

        # Assign customers where adjusted costs are negative
        # and the corresponding facility is open
        x = ((adjusted_costs < 0) & (y[np.newaxis, :] == 1)).astype(int)

        # Return the Lagrangian solution
        return UFL_Solution(y, x, self.instance)


    def convertToFeasibleSolution(self,lagr_solution):
        """
        Method that, given the Lagrangian Solution computes and returns
        a feasible solution (as a UFL_Solution)
        """

        # Copy facility decisions from the Lagrangian solution
        y = lagr_solution.y.copy()

        # If no facility is open, select the cheapest facility
        # considering both opening and transportation costs
        if np.sum(y) == 0:
            total_costs = self.instance.f + np.sum(
                self.instance.c, axis=0
            )

            cheapest_facility = np.argmin(total_costs)
            y[cheapest_facility] = 1

        # Identify all open facilities
        open_facilities = np.where(y == 1)[0]

        # Find the cheapest open facility for each customer
        cheapest_assignments = open_facilities[
            np.argmin(self.instance.c[:, open_facilities], axis=1)
        ]

        # Construct the assignment matrix
        x = np.zeros(
            (self.instance.n_markets, self.instance.n_facilities),
            dtype=int
        )

        x[np.arange(self.instance.n_markets), cheapest_assignments] = 1

        # Return the repaired feasible solution
        return UFL_Solution(y, x, self.instance)


    def updateMultipliers(self, labda_old, lagr_solution):
        """
        Updates Lagrangian multipliers using Polyak-type
```

```
        subgradient ascent and returns a new array of Lagrangian multipliers
        """

        # Calculate assignment violations for each customer
        g = 1 - np.sum(lagr_solution.x, axis=1)

        # Squared norm of the subgradient
        g_norm_squared = np.sum(g ** 2)

        # Avoid division by zero
        if g_norm_squared == 0:
            return labda_old.copy()

        # Calculate the current Lagrangian lower bound
        currentLB = self.computeTheta(labda_old)

        # Obtain a feasible upper bound if none exists yet
        if not np.isfinite(self.bestUB):
            feasible_solution = self.convertToFeasibleSolution(
                lagr_solution
            )
            self.bestUB = feasible_solution.getCosts()

        # Calculate the Polyak step size
        step_size = (
            self.beta * max(0.0, self.bestUB - currentLB)
            / g_norm_squared
        )

        # Update multipliers
        labda_new = labda_old + step_size * g

        return labda_new


    def runHeuristic(self, max_iterations=1000, stagnation_limit=20, initial_beta=1.5):
        """Run the Lagrangian heuristic and record iteration diagnostics."""

        labda = np.min(self.instance.c, axis=1).astype(float)

        self.beta = initial_beta
        self.bestUB = float("inf")

        bestLB = float("-inf")
        best_solution = None

        stagnation_counter = 0
        history = []
        stop_reason = "Maximum iterations reached"

        for k in range(1, max_iterations + 1):

            # Compute current Lagrangian lower bound
            currentLB = self.computeTheta(labda)

            if currentLB > bestLB + 1e-9:
                bestLB = currentLB
                stagnation_counter = 0
            else:
                stagnation_counter += 1

            # Solve the Lagrangian relaxation
            lagr_solution = self.computeLagrangianSolution(labda)

            # Construct a feasible solution
            feasible_solution = self.convertToFeasibleSolution(lagr_solution)

            if not feasible_solution.isFeasible():
                raise RuntimeError("The repaired solution is infeasible")

            currentUB = feasible_solution.getCosts()

            if currentUB < self.bestUB:
                self.bestUB = currentUB
                best_solution = feasible_solution

            if bestLB > self.bestUB + 1e-7:
                raise RuntimeError("Lower bound exceeds upper bound")

            # Calculate relative optimality gap
            gap = (
                (self.bestUB - bestLB)
                / max(1.0, abs(self.bestUB))
            )

            # Calculate subgradient and its norm
            g = 1 - np.sum(lagr_solution.x, axis=1)
```

```
            g_norm_squared = float(np.sum(g ** 2))
            g_norm = float(np.sqrt(g_norm_squared))

            # Check termination conditions
            gap_reached = gap <= 1e-6
            zero_subgradient = g_norm_squared == 0

            # Reduce beta after prolonged stagnation
            if (
                not gap_reached
                and not zero_subgradient
                and stagnation_counter >= stagnation_limit
            ):
                self.beta *= 0.5
                stagnation_counter = 0

            # Calculate Polyak step size for diagnostics
            if gap_reached or zero_subgradient:
                step_size = 0.0
            else:
                step_size = (
                    self.beta
                    * max(0.0, self.bestUB - currentLB)
                    / g_norm_squared
                )

            # Record iteration results and multiplier diagnostics
            history.append({
                "iteration": k,
                "current_LB": currentLB,
                "current_UB": currentUB,
                "best_LB": bestLB,
                "best_UB": self.bestUB,
                "gap_percent": gap * 100,
                "beta": self.beta,
                "subgradient_norm": g_norm,
                "step_size": step_size,
                "multiplier_mean": float(np.mean(labda)),
                "multiplier_std": float(np.std(labda)),
                "multipliers": labda.copy()
            })

            # Stop if optimality gap is sufficiently small
            if gap_reached:
                stop_reason = "Optimality gap reached"
                break

            # Stop if subgradient is zero
            if zero_subgradient:
                stop_reason = "Zero subgradient"
                break

            # Update Lagrangian multipliers
            labda = self.updateMultipliers(labda, lagr_solution)

        return (bestLB, self.bestUB, best_solution, history, stop_reason)
```

## F.2 `main.py` – Computational Experiments

The file `main.py` executes the heuristic on benchmark instances MO1–MO5, records the lower and upper bounds, and generates the convergence plots and result files.

**Listing 2:** Benchmark execution and convergence visualization

```

from pathlib import Path
import csv
import os
import time

import numpy as np
import matplotlib.pyplot as plt

from UFL import UFL_Problem, LagrangianHeuristic


BASE_DIR = Path(__file__).resolve().parent
INSTANCE_DIR = BASE_DIR / "Instances"
RESULTS_DIR = BASE_DIR / "Results"

INSTANCE_NAMES = ["MO1", "MO2", "MO3", "MO4", "MO5"]

MAX_ITERATIONS = 5000
STAGNATION_LIMIT = 20
```

```python
INITIAL_BETA = 1.5


def load_instance(name):
    """Load and validate a UFL benchmark instance."""

    instance_path = INSTANCE_DIR / name

    if not instance_path.is_file():
        raise FileNotFoundError(
            f"Instance file not found: {instance_path}"
        )

    previous_dir = Path.cwd()

    try:
        os.chdir(BASE_DIR)
        instance = UFL_Problem.readInstance(name)
    finally:
        os.chdir(previous_dir)

    if instance.c.shape != (
        instance.n_markets,
        instance.n_facilities
    ):
        raise ValueError(
            "Invalid transportation cost matrix"
        )

    if instance.f.shape != (instance.n_facilities,):
        raise ValueError(
            "Invalid facility opening cost vector"
        )

    return instance


def plot_convergence(name, history):
    """Plot best bounds and annotate the final solution quality."""

    iterations = np.array([
        row["iteration"] for row in history
    ])

    bestLB = np.array([
        row["best_LB"] for row in history
    ])

    bestUB = np.array([
        row["best_UB"] for row in history
    ])

    finalLB = bestLB[-1]
    finalUB = bestUB[-1]

    final_gap = (
        (finalUB - finalLB)
        / max(1.0, abs(finalUB))
        * 100
    )

    fig, ax = plt.subplots(figsize=(11, 6))

    ax.step(
        iterations,
        bestLB,
        where="post",
        color="tab:blue",
        linewidth=2,
        label="Best lower bound"
    )

    ax.step(
        iterations,
        bestUB,
        where="post",
        color="tab:orange",
        linewidth=2,
        label="Best upper bound"
    )

    ax.scatter(
        iterations[-1],
        finalLB,
        color="tab:blue",
        s=45,
        zorder=5
```

```
    )

    ax.scatter(
        iterations[-1],
        finalUB,
        color="tab:orange",
        s=45,
        zorder=5
    )

    result_text = (
        f"Best lower bound: {finalLB:,.2f}\n"
        f"Best upper bound: {finalUB:,.2f}\n"
        f"Best solution cost: {finalUB:,.2f}\n"
        f"Relative gap: {final_gap:.2f}%"
    )

    ax.text(
        0.98,
        0.70,
        result_text,
        transform=ax.transAxes,
        fontsize=11,
        verticalalignment="top",
        horizontalalignment="right",
        bbox={
            "boxstyle": "round,pad=0.6",
            "facecolor": "white",
            "edgecolor": "lightgray",
            "alpha": 0.95
        }
    )

    ax.set_title(
        f"{name}: Lagrangian Heuristic Convergence",
        fontsize=15
    )

    ax.set_xlabel("Iteration", fontsize=12)
    ax.set_ylabel("Objective value", fontsize=12)

    ax.set_xlim(1, max(2, len(history)))

    ax.grid(True, alpha=0.3)
    ax.legend(loc="upper right")

    fig.tight_layout()

    figure_path = RESULTS_DIR / f"{name}_convergence.png"

    fig.savefig(
        figure_path,
        dpi=200,
        bbox_inches="tight"
    )

    plt.close(fig)


def save_history(name, history):
    """Save iteration results and Lagrangian multipliers."""

    # Save iteration statistics
    history_path = RESULTS_DIR / f"{name}_history.csv"

    scalar_fields = [
        key for key in history[0].keys()
        if key != "multipliers"
    ]

    with history_path.open("w", newline="") as file:

        writer = csv.DictWriter(
            file,
            fieldnames=scalar_fields
        )

        writer.writeheader()

        for row in history:
            writer.writerow({
                key: row[key] for key in scalar_fields
            })

    # Save multiplier values separately
    multiplier_path = RESULTS_DIR / f"{name}_multipliers.csv"

```

```
    n_customers = len(history[0]["multipliers"])

    with multiplier_path.open("w", newline="") as file:

        writer = csv.writer(file)

        writer.writerow(
            ["iteration"]
            + [f"lambda_{i+1}" for i in range(n_customers)]
        )

        for row in history:
            writer.writerow(
                [row["iteration"]]
                + row["multipliers"].tolist()
            )


def plot_multiplier_evolution(name, history, n_customers=8):
    """Plot selected Lagrangian multipliers over iterations."""

    iterations = np.array([
        row["iteration"] for row in history
    ])

    multipliers = np.array([
        row["multipliers"] for row in history
    ])

    # Select customers with the largest multiplier variation
    variation = np.std(multipliers, axis=0)

    selected = np.argsort(variation)[-n_customers:]

    fig, ax = plt.subplots(figsize=(11, 6))

    for i in selected:
        ax.plot(
            iterations,
            multipliers[:, i],
            linewidth=1.5,
            label=f"Customer {i+1}"
        )

    ax.set_title(
        f"{name}: Lagrangian Multiplier Evolution",
        fontsize=15
    )

    ax.set_xlabel("Iteration", fontsize=12)
    ax.set_ylabel("Multiplier value", fontsize=12)

    ax.grid(True, alpha=0.3)
    ax.legend(loc="best", fontsize=9, ncol=2)

    fig.tight_layout()

    fig.savefig(
        RESULTS_DIR / f"{name}_multipliers.png",
        dpi=200,
        bbox_inches="tight"
    )

    plt.close(fig)


def plot_subgradient_diagnostics(name, history):
    """Plot subgradient norm and Polyak step size."""

    iterations = np.array([
        row["iteration"] for row in history
    ])

    norms = np.array([
        row["subgradient_norm"] for row in history
    ])

    step_sizes = np.array([
        row["step_size"] for row in history
    ])

    fig, ax1 = plt.subplots(figsize=(11, 6))

    ax1.plot(
        iterations,
        norms,
        color="tab:blue",
```

```
        linewidth=1.5,
        label="Subgradient norm"
    )

    ax1.set_xlabel("Iteration", fontsize=12)
    ax1.set_ylabel(
        "Subgradient norm",
        color="tab:blue",
        fontsize=12
    )

    ax1.tick_params(axis="y", labelcolor="tab:blue")

    ax2 = ax1.twinx()

    ax2.plot(
        iterations,
        step_sizes,
        color="tab:orange",
        linewidth=1.5,
        alpha=0.8,
        label="Polyak step size"
    )

    ax2.set_ylabel(
        "Step size",
        color="tab:orange",
        fontsize=12
    )

    ax2.tick_params(axis="y", labelcolor="tab:orange")

    ax1.set_title(
        f"{name}: Subgradient and Step-Size Diagnostics",
        fontsize=15
    )

    ax1.grid(True, alpha=0.3)

    lines1, labels1 = ax1.get_legend_handles_labels()
    lines2, labels2 = ax2.get_legend_handles_labels()

    ax1.legend(
        lines1 + lines2,
        labels1 + labels2,
        loc="upper right"
    )

    fig.tight_layout()

    fig.savefig(
        RESULTS_DIR / f"{name}_subgradient.png",
        dpi=200,
        bbox_inches="tight"
    )

    plt.close(fig)


def main():
    """Run all benchmark instances and report their results."""

    RESULTS_DIR.mkdir(exist_ok=True)

    summary = []
    total_start = time.perf_counter()

    print("\nLagrangian Heuristic Benchmark Results")
    print("=" * 112)

    header = (
        f"{'Instance':<10}"
        f"{'Best LB':>14}"
        f"{'Best UB':>14}"
        f"{'Gap (%)':>12}"
        f"{'Iterations':>12}"
        f"{'Open Fac.':>12}"
        f"{'Time (s)':>12}"
        f"  {'Termination'}"
    )

    print(header)
    print("-" * 112)

    for name in INSTANCE_NAMES:

        instance = load_instance(name)
```

```
        heuristic = LagrangianHeuristic(instance)

        start_time = time.perf_counter()

        (   bestLB,
            bestUB,
            best_solution,
            history,
            stop_reason
        ) = heuristic.runHeuristic(
            max_iterations=MAX_ITERATIONS,
            stagnation_limit=STAGNATION_LIMIT,
            initial_beta=INITIAL_BETA
        )

        execution_time = time.perf_counter() - start_time

        if best_solution is None:
            raise RuntimeError(
                "No feasible solution was found"
            )

        if not best_solution.isFeasible():
            raise RuntimeError(
                "The final solution is infeasible"
            )

        gap_percent = ((bestUB - bestLB) / max(1.0, abs(bestUB)) * 100)

        plot_convergence(name, history)
        plot_multiplier_evolution(name, history)
        plot_subgradient_diagnostics(name, history)
        save_history(name, history)

        open_facilities = int(np.sum(best_solution.y))

        result = {
            "instance": name,
            "best_LB": bestLB,
            "best_UB": bestUB,
            "gap_percent": gap_percent,
            "iterations": len(history),
            "open_facilities": open_facilities,
            "time_seconds": execution_time,
            "stop_reason": stop_reason
        }

        summary.append(result)

        print(
            f"{name:<10}"
            f"{bestLB:>14,.4f}"
            f"{bestUB:>14,.4f}"
            f"{gap_percent:>12.4f}"
            f"{len(history):>12}"
            f"{open_facilities:>12}"
            f"{execution_time:>12.4f}"
            f"  {stop_reason}"
        )

    summary_path = RESULTS_DIR / "summary.csv"

    with summary_path.open("w", newline="") as file:

        writer = csv.DictWriter(file, fieldnames=list(summary[0].keys()))

        writer.writeheader()
        writer.writerows(summary)

    total_time = time.perf_counter() - total_start

    print("=" * 112)
    print(f"Total execution time: {total_time:.4f} seconds")
    print(f"Maximum iterations: {MAX_ITERATIONS}")
    print(f"Initial beta: {INITIAL_BETA}")
    print(f"Stagnation limit: {STAGNATION_LIMIT}")
    print(f"Results directory: {RESULTS_DIR}")
    print("\nAll benchmark experiments completed.")


if __name__ == "__main__":
    main()
```